

사용자 취향과 실시간 맥락을 반영한 AI 여행 플래너 에이전트 최종보고서



팀명: 고준이

고태영 학번: 202055510 이메일: koty08@pusan.ac.kr

박준이 학번: 202155556 이메일: zun_e@pusan.ac.kr

지도교수: 탁성우

제출일: 2026년 9월 18일

목 차

1	서론	1
1.1	연구 동기	1
1.2	기존 문제점	1
1.3	연구 목표	2
1.4	연구 수행 경과와 범위 조정	2
2	연구 배경	4
2.1	검색 증강 생성과 검색 보정	4
2.2	벡터 검색과 개인화	4
2.3	LLM 오케스트레이션과 결정적 안전장치	4
2.4	평가 하네스와 실시간 애플리케이션	5
3	연구 내용	6
3.1	요구사항 진화와 최종 설계 결정	6
3.2	전체 시스템 구조	7
3.2.1	모노레포 구성	8
3.2.2	배포 토폴로지	9
3.3	사용자 기능 구현	9
3.3.1	인증과 계정	10
3.3.2	취향 입력과 사진 분석	10
3.3.3	여행 생성, 협업, 일정 편집	11
3.3.4	웹 · 모바일 통합	12
3.4	장소 데이터와 CRAG 검색	13
3.4.1	카탈로그 적재 파이프라인	13
3.4.2	후보 검색과 보정 흐름	13
3.4.3	점수 향과 후보 구성	14
3.5	AI 일정 생성과 제약 검증	14
3.5.1	일정 생성 알고리즘	14

3.5.2	하드 제약 엔진	15
3.5.3	결정적 폴백이 실패를 가리는 문제	16
3.6	알림과 재계획	16
3.6.1	알림 우선 정책	16
3.6.2	비동기 잡과 실시간 결과	18
3.7	데이터 모델과 외부 도구	18
3.7.1	핵심 데이터 모델	18
3.8	보안, 운영 안정성, 관측성	19
3.8.1	계정 탈취 경로 차단	19
3.8.2	운영 안정성과 관측성	20
4	연구 결과 분석 및 평가	22
4.1	개발 수행 체계와 역할 분담	22
4.2	착수 · 중간 · 최종 요구사항 추적	23
4.3	최종 구현 결과 요약	25
4.4	코드 품질과 빌드 검증	26
4.5	착수 단계 평가 목표 대비 달성도	27
4.6	검색 품질 평가	29
4.7	실행 성능과 시스템 검증	30
4.8	한계와 타당성 위협	31
5	결론 및 향후 연구 방향	32
5.1	결론	32
5.2	연구 기여	32
5.3	향후 연구 방향	33
6	참고 문헌	34

1. 서론

1.1. 연구 동기

여행 일정 수립은 목적지 검색, 운영시간 확인, 이동시간 계산, 날씨 확인, 동행자 의견 조율을 동시에 다루어야 하는 복합 의사결정 문제이다. 기존 지도나 관광 정보 서비스는 장소 단위의 검색 결과를 제공하는 데 강점이 있지만, 사용자의 취향과 여행 기간, 기상·혼잡·현재 위치를 함께 고려하여 하루 단위의 실행 가능한 일정으로 조합하는 과정은 사용자에게 남아 있다. 특히 일정이 시작된 후에는 비 예보, 관광지 혼잡, 예정 장소 미도착과 같은 변화가 발생하므로 초기 추천만으로 여행 전 과정을 지원하기 어렵다.

생성형 AI와 검색 증강 생성(Retrieval-Augmented Generation, RAG)은 자연어로 표현된 취향과 구조화된 장소 정보를 결합할 수 있게 한다[1]. 그러나 언어 모델이 장소를 임의로 생성하거나 영업시간·이동시간을 위반하는 문제를 해결하지 않으면, 문장 품질이 좋아도 실제 여행 일정으로 사용할 수 없다. 본 연구는 이 문제를 소프트웨어 시스템 관점에서 다루기 위해 검색, 검색 결과 평가, 일정 생성, 하드 제약 검증, 결정적 폴백을 하나의 파이프라인으로 통합한 AI 여행 플래너 에이전트 **TriPick**을 구현하였다.

TriPick은 Next.js 웹 애플리케이션과 React Native 모바일 셀, NestJS API 서버, PostgreSQL/pgvector, Redis/BullMQ, OpenAI-compatible 로컬 LLM 추론 서버로 구성된다. 사용자는 음식·분위기·환경 취향을 직접 선택하거나 사진 분석으로 보강하고, 여행 지역·기간·기상/취침 시간·이동 수단을 입력해 일정을 생성한다. 여행 중 이벤트는 자동으로 일정을 덮어쓰지 않고 알림으로 전달되며, 사용자가 확인한 뒤 전체 또는 특정 일차의 재계획을 요청한다.

1.2. 기존 문제점

본 프로젝트가 해결하려는 기존 문제는 다음 네 가지로 정리된다.

- 취향 정보의 단절:** 사용자가 선호하는 음식, 분위기, 자연/도심 성향이 장소 검색과 일정 배치에 일관되게 연결되지 않는다.
- 검색 결과 품질의 불균일:** 벡터 유사도만으로는 지역 대표 장소, 대중 인지도, 현재 이벤트 맥락, 영업 가능 여부를 충분히 구분하지 못한다.
- 생성 결과의 실행 불가능성:** LLM은 존재하지 않는 장소를 만들거나, 취침 이후 일정, 영업시간 위반, 비현실적인 이동 간격을 생성할 수 있다.
- 실시간 변화 처리의 과도한 자동화:** 위치 이탈이나 날씨 변화가 감지될 때 사용자 동의 없이 일정을 변경하면 기존 약속과 메모가 사라지고, 재계획 결과를 신뢰하기 어렵다.

따라서 본 연구는 “추천 정확도” 하나만 최적화하지 않는다. 후보 데이터의 지역 정합성, 카테고리 균형, 일정 하드 제약, 기존 데이터 보존, 실패 시 복구 경로까지 포함해 서비스를 끝까지 사용할 수 있는가를 핵심

품질 기준으로 설정하였다.

1.3. 연구 목표

최종 목표는 사용자의 취향과 여행 조건을 반영한 국내 여행 일정을 생성하고, 여행 중 맥락 변화가 생겼을 때 사용자의 확인을 거쳐 안전하게 재계획하는 통합 시스템을 완성하는 것이다. 구체적인 목표는 다음과 같다.

- 수동 선택과 사진 분석 결과를 공통 취향 프로필 및 1,024차원 임베딩으로 관리한다.
- 관광공사 · 카카오 장소 데이터를 정규화해 pgvector 카탈로그로 적재하고, 지역 pre-filter와 벡터 검색을 결합한다.
- CRAG 방식의 검색 평가기를 구현해 유사도, 취향, 인지도, 지역성, 이벤트 맥락, 가용성을 점수화한다 [2].
- LLM이 후보 ID만 사용하도록 제한하고, 기상/취침 시간, 영업시간, 이동 ETA를 하드 제약으로 검증한다.
- LLM 실패나 제약 위반 시 근접 후보 기반 결정적 재생성을 최대 3회 수행하고, 유효한 안이 없으면 기존 일정을 보존한다.
- WebSocket, inbox, FCM을 통해 알림과 재계획 결과를 전달하되, 자동 재계획 대신 사용자 승인 흐름을 유지한다.
- 웹 · 모바일 · API · 데이터 · AI 추론을 배포 가능한 모노레포와 반복 가능한 테스트로 통합한다.

1.4. 연구 수행 경과와 범위 조정

본 연구는 2026년 4월 29일 착수보고서, 7월 8일 중간보고서, 8월 31일 최종 코드 분석의 세 시점을 거쳤다. 착수보고서는 해결할 문제와 목표 기술 구조, WBS, 평가 목표치를 정의했고, 중간보고서는 develop 브랜치의 ded5d37 커밋을 기준으로 실제 구현에서 발생한 요구조건 조정과 잔여 작업을 정리했다. 본 최종보고서는 이 두 문서를 단순 인용하는 데 그치지 않고, 각 시점의 판단이 최종 코드에서 어떻게 구현 · 변경 · 유보되었는지를 추적한다. 표 1.1는 세 시점의 기준일과 각 문서가 맡는 역할을 정리한 것이다.

착수 단계의 핵심 방향은 취향 기반 일정 생성, 외부 도구 호출, CRAG 검색 보정, 하드 제약 검증, 실시간 재계획, LLM harness, 프라이빗 추론 인프라였다. 중간 단계에서는 제한된 기간 안에 시연 가능한 핵심 흐름을 완성하기 위해 Instagram 자동 수집과 완전 자동 재계획을 후순위로 내리고, 수동 취향 입력 · 직접 사진 업로드, 카드/타임라인 중심 UI, 사용자 확인형 재계획, OpenAI-compatible adapter를 우선하였다. 최종 구현은 이 조정 원칙을 유지하면서 장소 카탈로그와 검색 평가, 결정적 제약 검증, 알림 · 재계획, 배포 안정성을 추가로 고도화했다.

표 1.1. 보고 단계별 기준과 역할

단계	기준일/코드	문서의 역할	최종보고서 반영 방식
착수보고서	2026.04.29	목표, MVP 범위, WBS, 역할, 평가 목표 정의	최초 요구사항과 수치 목표의 기준선
중간보고서	2026.07.08 / ded5d37	요구조건 수정, 설계 상세화, 진척도, 잔여 위험 기록	변경 사유와 미완료 항목의 기준선
최종 코드	2026.08.31 / 50423ca	실제 구현, 운영 문서, 테스트, 프로덕션 데이터의 정본	구현 여부와 한계를 코드 · 실측으로 판정

2. 연구 배경

2.1. 검색 증강 생성과 검색 보정

RAG는 외부 지식원에서 검색한 문서를 생성 모델의 입력으로 제공하여 모델 파라미터에만 의존하는 생성의 한계를 줄인다[1]. 여행 플래닝에서는 장소 카탈로그를 검색해 모델이 사용할 후보를 제한할 수 있다는 장점이 있다. 그러나 검색 결과가 부정확하면 생성 모델도 잘못된 후보를 사용한다. Corrective RAG는 검색 결과를 평가하고 필요할 때 보정 검색을 수행하는 관점을 제시한다[2].

TriPick은 이 관점을 장소 추천에 맞게 변형하였다. pgvector 코사인 검색 결과를 바로 LLM에 넣지 않고, 후보마다 다음 점수를 계산한다.

$$S(p) = w_r S_{retrieval}(p) + w_t S_{taste}(p) + w_p S_{popularity}(p) + w_l S_{locality}(p) + w_c S_{context}(p) + w_a S_{availability}(p). \quad (2.1)$$

현재 코드 기본값은 $w_r = 0.06$ 이고, 나머지 항은 상대비 0.20 : 0.12 : 0.16 : 0.13 : 0.09를 $1 - w_r$ 에 비례 배분한다. 따라서 실효 가중치는 취향 0.269, 인지도 0.161, 지역성 0.215, 맥락 0.175, 가용성 0.121이다. 벡터 유사도가 후보 풀 안에서 좁은 범위에 몰리는 현상을 실측한 뒤, 유사도 항을 중복 강조하지 않도록 조정된 결과이다.

2.2. 벡터 검색과 개인화

PostgreSQL 확장인 pgvector는 관계형 데이터와 벡터를 한 DB에서 관리하고 코사인 거리 기반 인덱스를 제공한다[3]. TriPick은 preference_embeddings와 place_embeddings를 각각 vector(1024)로 구성하고 HNSW 인덱스를 둔다. 질의 벡터 q 와 사용자 취향 벡터 u 는 다음과 같이 결합한다.

$$q' = \text{normalize}(0.85q + 0.15u). \quad (2.2)$$

개인화는 식 2.2 한 경로에만 의존하지 않는다. 질의 텍스트에도 음식 · 분위기 · 환경 태그가 들어가며, CRAG 취향 점수는 태그 일치와 사용자 벡터 유사도를 다시 결합한다. 차원이 다르면 검색 전체가 실패하지 않도록 개인화 벡터를 건너뛰고 재임베딩 필요 로그를 남긴다.

2.3. LLM 오케스트레이션과 결정적 안전장치

TriPick의 Planner는 LLM이 스스로 도구를 선택하는 완전 자율 에이전트가 아니다. NestJS의 Planner 서비스는 검색, 날씨, 경로, LLM 호출, 제약 검증 순서를 코드로 결정한다. LLM은 정해진 후보 안에서 일차와

순서, 체류시간, 간단한 이유를 JSON으로 배치하는 역할을 맡는다. 이 구조는 모델이 불안정하거나 추론 서버가 없을 때에도 동일한 입력으로 결정적 폴백을 실행할 수 있다는 장점이 있다.

LLM 서빙은 OpenAI-compatible /chat/completions 인터페이스를 사용한다. vLLM과 같은 서빙 시스템은 GPU 메모리 효율과 요청 처리량을 높일 수 있으며[4], 프로젝트는 현재 llama.cpp 기반 chat/vision 서버와 embedding 서버를 한 RunPod에 분리 포트로 운영한다. chat/vision은 gemma-4-26B-A4B-it(Q4 양자화, 총 26B 중 활성 4B의 MoE)와 멀티모달 projector를 :8080에, embedding은 BGE-m3-ko(f16, 1,024 차원, CLS 풀링)를 :8081에 올린다. VRAM 점유는 각각 약 16 GB, 1.5 GB, 1.2 GB로 24 GB 카드 한 장에 모두 올라가므로 pod를 나눠 GPU 비용을 두 번 내지 않아도 된다. 모델 호출 기본 타임아웃은 90 초, temperature는 0.2이다. 응답이 없거나 JSON이 유효하지 않으면 후보 카테고리나 시간 슬롯을 이용해 결정적으로 일정을 구성한다.

2.4. 평가 하네스와 실시간 애플리케이션

RAG 시스템 평가는 생성 문장만이 아니라 검색 문맥의 관련성과 충실성을 함께 봐야 한다. RAGAS는 이러한 자동 평가 관점을 제공한다[5]. TriPick은 이를 서비스 규모에 맞게 축소해 16개 대표 검색 케이스의 recall@5, recall@10, 카탈로그 내 recall, MRR, 지역 정합, 금지어, 영업시간 위반을 측정한다. 인지도 데이터가 네이버 검색 스니펫에 의존하므로, 시간에 따른 코퍼스 변동도 평가 한계로 기록한다.

클라이언트는 Next.js App Router[6]와 React Native[7]를 사용한다. WebView가 주요 화면을 제공하고 네이티브 계층은 위치 권한, FCM, Secure Storage, 외부 링크를 담당한다. 서버는 NestJS 모듈 구조[8], BullMQ 비동기 잡[9], Socket.IO 실시간 채널[10]로 API와 백그라운드 처리를 분리한다.

3. 연구 내용

3.1. 요구사항 진화와 최종 설계 결정

착수보고서의 방향은 유지되었지만, 구현 과정에서 외부 심사 의존성, 데이터 품질, 모바일 사용성, GPU 운영비를 확인하면서 일부 요구사항의 수단과 우선순위가 바뀌었다. 중간보고서는 이 변화를 “기능 축소”가 아니라 핵심 흐름의 완성도를 높이기 위한 재범위화로 정의하였다. 최종 코드 분석에서도 같은 원칙을 적용해, 계획과 다른 구현을 누락으로만 보지 않고 문제를 더 안전하게 해결하는지와 검증 근거가 있는지를 함께 판단하였다. 표 3.1은 영역별로 착수 계획, 중간 조정, 최종 결정을 나란히 놓아 그 변화를 보인다.

표 3.1. 착수 · 중간보고서 대비 최종 설계 변화

영역	착수 단계 계획	중간 단계 조정	최종 설계 결정
인증	Kakao 로그인 중심	외부 OAuth 실패에 대비해 이메일 가입 · 인증 · 재설정과 refresh session을 우선	이메일/Kakao 병행, refresh rotation, 모바일 Secure Storage, 탈퇴 · 토큰 폐기까지 구현
취향 입력	Instagram 사진 수집과 직접 업로드로 이미지 취향 분석	Graph API 심사 · 권한 위험 때문에 수동 입력과 직접 업로드를 MVP 정보으로 전환	구조화 취향 + private 사진 + Vision tag + 1,024차원 재임베딩 구현; Instagram 자동 수집은 제외
검색/추천	pgvector RAG 후 LLM 일정 생성	CRAG evaluator, 취향 vector blending, fallback, constraint 검증으로 세분화	지역 pre-filter, 목적지 앵커, 6항 점수, 카테고리 quota/상한, Kakao fallback을 결합
실시간 변화	웨이팅 · 경로 이탈 · 우전을 감지해 자동 대안 제안	Live 화면과 수동 waiting/deviation 신고, WebSocket result를 우선	신뢰 가능한 데이터가 없는 waiting 직접 입력은 제거; 날씨 · 혼잡 · 미도착 알림 후 사용자가 범위를 확인해 재계획
지도/화면	지도 중심의 상세 여행 화면	모바일 사용성을 위해 카드/타임라인과 Live 화면을 우선	지도는 목적지 · 현재 위치 확인에 집중하고 일정 편집 · 진행은 카드/타임라인으로 제공

다음 쪽에 계속

표 3.1. 착수 · 중간보고서 대비 최종 설계 변화(계속)

영역	착수 단계 계획	중간 단계 조정	최종 설계 결정
LLM 인프라	Gemma 4/llama.cpp 에서 vLLM · Triton 으로 확장	OpenAI-compatible adapter로 외부/프라이빗 endpoint와 결정적 fallback을 교체 가능하게 구성	RunPod 한 pod의 llama.cpp chat/vision(:8080) · embedding(:8081) 채택; vLLM/Triton은 성능 실험 항목으로 유보
재계획 안전성	이벤트에 반응하는 대안 동선	사용자 확인과 실패 시 기존 일정 보존을 명시	완료 항목 · 메모 · 비대상 일차를 보존하고, 유효한 새 안이 없으면 DB를 덮어쓰지 않는 부분 재계획 구현

이 변화에서 가장 중요한 결정은 “알림”과 “일정 변경”을 분리한 것이다. 착수 단계의 자동 재계획은 데모상 매력적이지만 위치 오차, 혼잡 데이터 결손, 사용자의 약속과 메모를 훼손할 수 있다. 최종 시스템은 스케줄러가 날씨 · 혼잡 · 미도착 조건을 감지해 inbox/FCM/WebSocket으로 알리고, 사용자가 재계획 범위와 조건을 확인한 뒤 BullMQ 작업을 생성한다. 이는 중간보고서의 시연 우선 원칙을 운영 안전성까지 확장한 결과이다.

3.2. 전체 시스템 구조

그림 3.1는 최종 코드의 실행 계층을 나타낸다. 클라이언트는 REST와 WebSocket으로 NestJS API에 연결되고, API는 Planner 오케스트레이터를 통해 검색 · LLM · 제약 엔진을 호출한다. PostgreSQL은 업무 데이터와 벡터 데이터를 함께 보관하며, Redis는 캐시 · 레이트리밋 · BullMQ를 공유한다. 외부 서비스는 장소, 경로, 날씨, 혼잡, 푸시 정보를 제공한다.

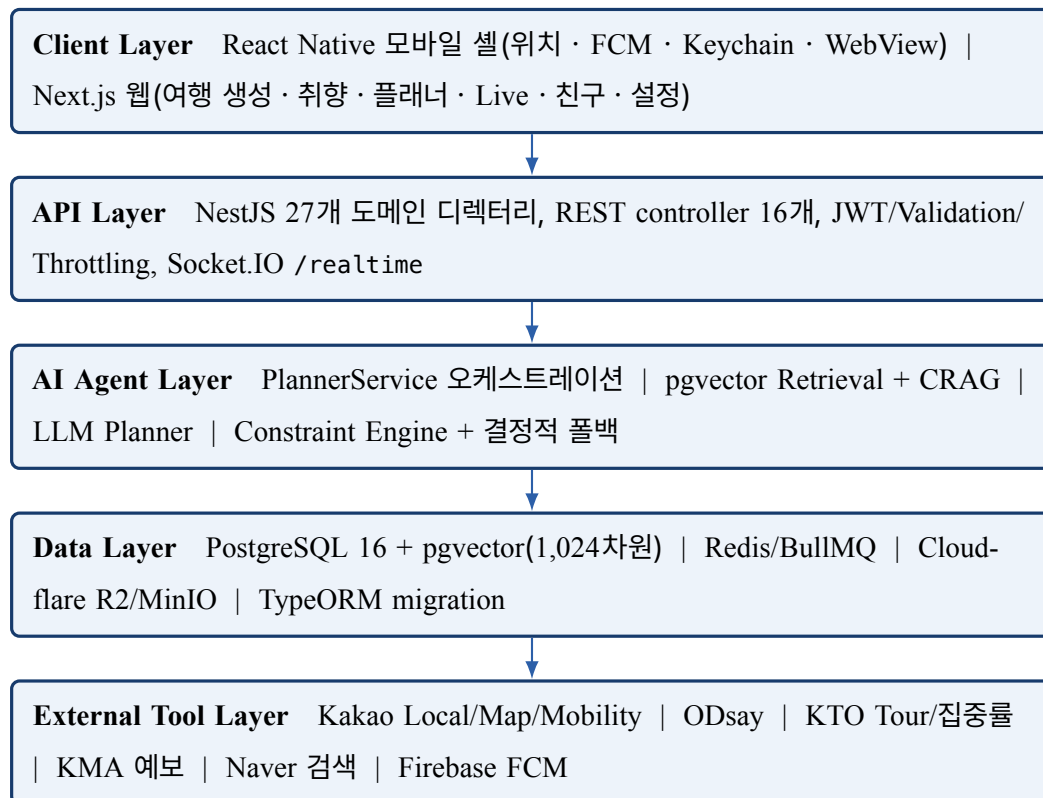


그림 3.1. TriPick 최종 시스템 아키텍처

3.2.1. 모노레포 구성

프로젝트는 pnpm workspace와 Turborepo로 구성된다. packages/types는 클라이언트와 서버가 공유하는 DTO · 이벤트 · 도메인 타입을, packages/utis는 날짜, 좌표 변환, 하버사인 거리, 기상 파서를 제공한다. 표 3.2의 규모는 단순 계획이 아니라 실제 최종 소스의 TypeScript/TSX 행 수를 집계한 결과이다.

표 3.2. 모노레포 코드 구성

경로	파일	행 수	주요 역할
apps/api/src	193	28,717	NestJS API, AI Planner, 검색/적재, 알림, 인증, 협업
apps/web/src	260	22,620	19개 page route, 플래너/지도/취향/Live/친구/인박스 UI
apps/mobile/src	1	902	React Native WebView 셀, 네이티브 브리지, FCM/위치/토큰
packages/types/src	15	1,352	공유 DTO, 이벤트, 도메인 타입
packages/utils/src	8	716	날짜, 지리, 기상 파서, 활동 시간 창
합계	477	54,307	모노레포 공통 빌드 · 타입체크 대상

3.2.2. 배포 토폴로지

웹은 Vercel, API/PostgreSQL/Redis는 Railway, 추론은 RunPod, 객체 저장소는 Cloudflare R2에 배치한다. 개발 환경에서는 Docker Compose가 pgvector PostgreSQL, Redis, MinIO, Mailpit을 제공한다. 프로덕션 API 이미지는 Node 20 slim의 비루트 사용자로 실행하며, 부팅 시 TypeORM migration을 적용한다[11]. 그림 3.2은 이 배치와 구성 요소 간 연결이다.

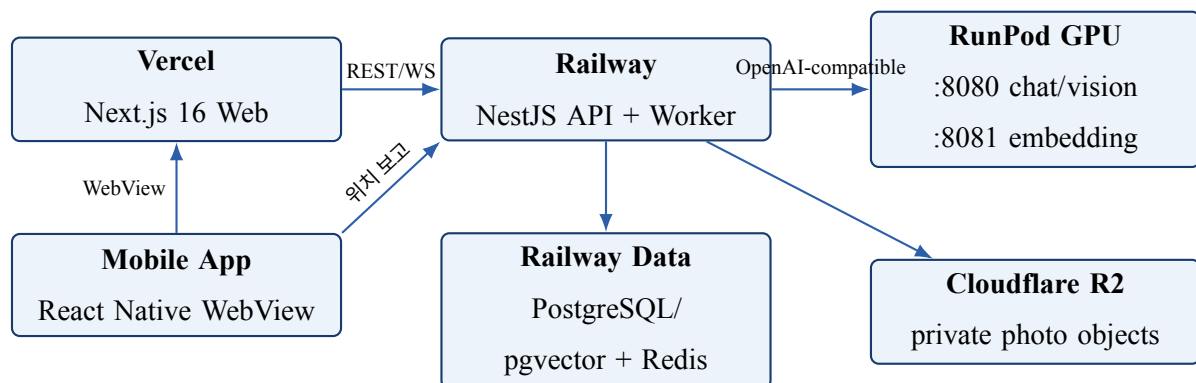


그림 3.2. 프로덕션 배포 토폴로지

3.3. 사용자 기능 구현

3.3.1. 인증과 계정

인증은 이메일 가입 · 로그인 · 메일 인증 · 비밀번호 재설정과 Kakao OAuth를 모두 제공한다. access token은 짧은 API 인증에 사용하고, refresh token은 family 단위로 회전하며 해시만 DB에 저장한다. 모바일은 refresh token을 Keychain/Keystore에 보관하고, WebView와 access token만 교환한다. 서버는 전역 ValidationPipe의 whitelist와 forbidNonWhitelisted를 적용하고, Redis 기반으로 IP당 60초 120회의 기본 레이트리밋을 둔다. 프로덕션에서는 기본 JWT secret을 사용하면 부팅을 거부한다.

계정 기능은 프로필 이미지, 알림 카테고리 설정, 비밀번호 변경, 회원 탈퇴 사유 수집을 포함한다. 취향 사진과 프로필 이미지는 private object로 저장하고 15분 만료 서명 URL을 사용한다. 카카오 OAuth는 state를 검증하고, 이메일 가입은 가입 메일 발송량과 토큰 만료를 제한한다.

3.3.2. 취향 입력과 사진 분석

사용자는 음식, 여행 분위기, 환경, 여행 속도, 활동 강도, 혼잡 선호를 직접 선택할 수 있다. 사진 업로드는 최대 크기와 MIME type을 검증한 뒤 private storage에 저장하고 BullMQ preference-analysis queue로 보낸다. Vision Analyzer는 사진별 음식 · 분위기 · 환경 태그와 confidence를 생성하고, 여러 사진 결과를 집계한다. 사용자는 사진별 태그를 켜거나 끌 수 있으며, 최종 preference text가 바뀌면 취향 임베딩을 다시 저장한다.

그림 3.3는 사진 분석이 끝난 취향 화면이다. 상단은 갤러리에서 고른 사진과 저장 범위 고지이고, 하단은 Vision Analyzer가 두 장에서 뽑아 집제한 태그(카페 · 한식 · 힐링 · 트렌디 · 문화 · 역사 · 도시 · 자연 · 산)와 사진별 태그 on/off 토글이다. 사용자가 태그를 끄면 preference text가 바뀌고 취향 임베딩이 다시 저장된다.

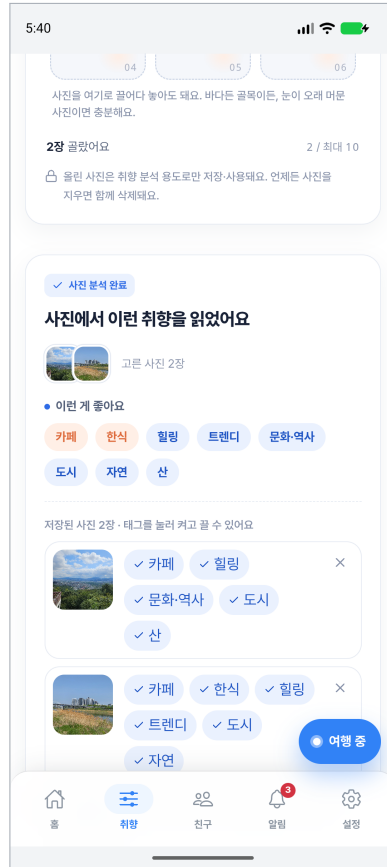


그림 3.3. 사진 취향 분석 결과와 사진별 태그 on/off 화면

Instagram Graph API는 권한 심사와 운영 정책 의존성이 높아 최종 MVP의 필수 입력에서 제외했다. 따라서 사진 분석이 실패해도 수동 취향 선택만으로 일정 생성이 가능하다. 이는 외부 플랫폼 실패가 핵심 플로우를 막지 않게 한 범위 조정이다.

3.3.3. 여행 생성, 협업, 일정 편집

개인화는 일정 생성 단계에서 시작되지 않는다. 여행 생성 화면에 노출되는 추천 목적지 (GET /destinations/recommended)부터 사용자 취향 벡터로 랭킹한다. 먼저 네이버 검색에서 이번 달 “국내 여행지 추천” 코퍼스를 모아 실제로 언급된 목적지만 후보로 추리고, 각 후보의 취향 점수와 계절 언급 빈도를 0.7:0.3으로 결합한다. 취향 점수는 place_embeddings에서 지역·시군구별 상위 12개 장소의 취향 벡터 코사인 유사도를 평균한 값이며, 장소가 5개 미만인 지역은 표본이 얇아 제외한다. 온보딩 전이라 취향 벡터가 없으면 계절 언급 순으로만 정렬하고, 네이버 키가 없거나 코퍼스가 비면 순수 취향 기반 지역 추천으로 폴백한다. 즉 취향 신호가 어느 단계에서 빠지든 추천 자체는 중단되지 않고 한 단계씩 중립으로 물러난다.

여행 생성 화면은 목적지 자동완성/지도 선택, 시작·종료일, 기상·취침 시각, 이동 수단, 일자별 복수 지역, 동행자를 입력받는다. 서버는 여행 기간을 계산하고 trip_days로 일차별 지역을 분리한다. 한 여행에 여러 지역이 있으면 Planner가 일차별 후보 풀을 독립적으로 검색하여 지역이 섞이는 문제를 막는다.

일정 화면은 장소별 예정 시각, 체류시간, 이동시간, 영업시간, 이미지, 사용자 메모를 제공한다. 사용자는

항목 추가 · 수정 · 삭제 · 드래그 재정렬, 대안 장소 교체, 전체/일차별 재계획을 수행할 수 있다. 재계획 시 같은 장소가 다시 배치되면 기존 사용자 메모를 보존하며, 오늘 이미 끝난 항목도 삭제하지 않는다. 여행 공유 토큰, 친구, 여행 멤버, 초대, 참여자 취향 조율, 소유자 일정 변경 승인 제안도 구현되어 있다.

그림 3.4는 이 세 화면이다. 왼쪽 플래너는 카카오프와 일차 타임라인을 함께 두어 장소별 예정 시각과 체류시간을 보여주고 드래그 재정렬 · 대안 교체(“변경”)를 제공한다. 가운데 Live 화면은 현재 위치와 다음 장소까지의 거리 · 예상 이동시간을 표시한다. 오른쪽 조율 탭은 참여자 3명의 취향 태그를 모아 항목별로 누가 무엇을 원하는지 비교한다.

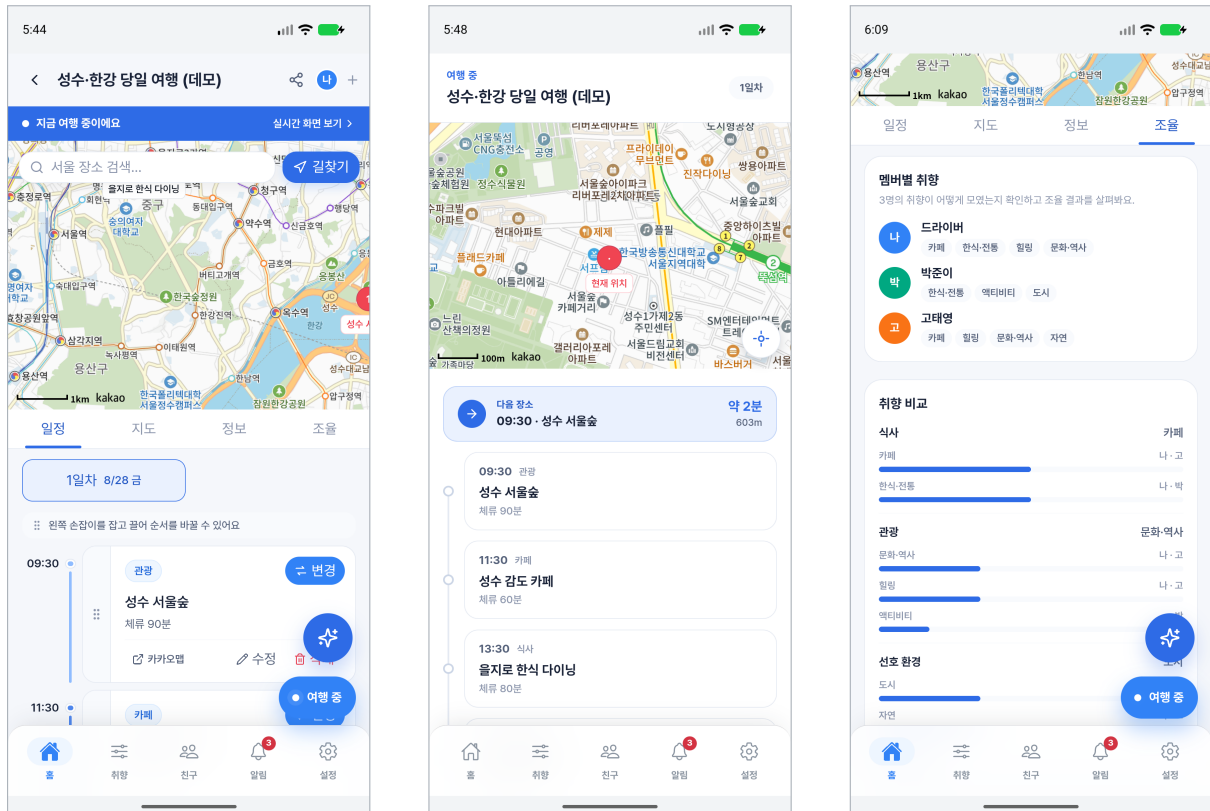


그림 3.4. (좌) 플래너 일정 · 지도, (중) 여행 중 Live 화면, (우) 참여자 취향 조율

3.3.4. 웹 · 모바일 통합

웹은 랜딩, 로그인/가입, 비밀번호 찾기 · 재설정, 이메일 인증, Kakao 콜백, 취향, 여행 목록/생성, 플래너, Live, 인박스, 친구, 설정, 지원, 약관 · 개인정보, 공유 일정의 19개 page route를 제공한다. 모바일 셀은 WebView 위에 다음 네이티브 기능을 연결한다.

- 위치 권한과 주기적 위치 보고. 백그라운드에서는 WebView의 JavaScript가 정지하므로, RN 네이티브가 foreground service로 잡은 좌표를 웹을 거치지 않고 API에 직접 전송한다
- Firebase/Notiffee 기반 push 수신과 딥링크
- Secure Storage 기반 refresh token 보관

- 외부 브라우저, 파일 공유, 네트워크/세션 상태 브리지

이 구조는 대부분의 UI를 Next.js에서 일관되게 개발하면서, 운영체제 권한이 필요한 기능만 React Native가 담당하도록 역할을 분리한다.

3.4. 장소 데이터와 CRAG 검색

3.4.1. 카탈로그 적재 파이프라인

장소 적재는 KTO 관광정보, Kakao category/keyword 검색, Naver 추천 글에서 추출한 인기 장소, 수동 keyword 소스를 지원한다[12], [13]. 입력 데이터는 지역 정보 코드, 이름 품질, 카테고리, 좌표, 행사 기간, 영업시간을 정규화한다. 숙박 · 소매 점포 · 행정구역 이름 · SEO성 상호 · 여행코스 기사는 일정 부적합 게이트에서 제외한다.

중복 판정은 외부 ID뿐 아니라 정규화 이름과 좌표 반경을 함께 사용한다. 같은 장소의 KTO 표현과 Kakao 표현이 실행마다 서로를 덮어써 재임베딩되는 churn을 막기 위해, 먼저 적재된 표현을 정보원으로 유지하고 비어 있는 영업시간만 보강한다. text_hash와 embedding model이 같으면 임베딩 호출을 생략하며, append cursor는 실행별 페이지 크기에 흔들리지 않도록 행 offset으로 저장한다.

3.4.2. 후보 검색과 보정 흐름

그림 3.5는 최종 검색 흐름이다. 행정구역으로 해석되지 않는 “광안리”, “서면역” 같은 목적지는 Kakao 결과로 좌표 앵커를 찾고 2 km, 5 km, 10 km 순으로 반경을 확장한다. 후보 수가 목표의 2배 이상이고 식음/비식음이 각각 4개 이상이며 카페가 포함될 때 반경을 확장한다. 그래도 얇으면 가장 넓은 반경 후보를 유지한 채 지역 전체 후보를 덧붙인다.

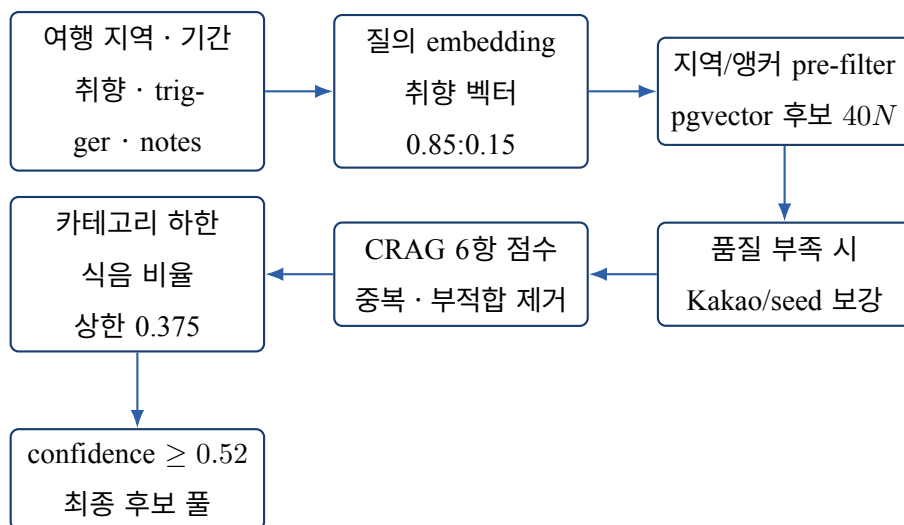


그림 3.5. pgvector 검색과 CRAG 보정 파이프라인

3.4.3. 점수 항과 후보 구성

표 3.3는 코드 기본값과 역할을 정리한다. 검색 결과의 평균 confidence가 0.61보다 낮거나 후보가 부족하면 Kakao 폴백을 수행하고, 최종 후보는 보정 confidence 0.52를 기준으로 선택한다. 인지도는 순위를 조정하는 소프트 신호이므로 언급이 없다는 이유만으로 후보를 제거하지 않는다. 네이버 코퍼스가 후보 이름을 충분히 언급하지 못한 지역에서는 인지도 항 자체를 중립으로 둔다.

표 3.3. CRAG 및 플래너 주요 코드 기본값

설정/상수	값	코드상 의미
CRAG_RETRIEVAL_WEIGHT	0.06	벡터 유사도 항; 나머지 0.94는 다른 5항에 비례 배분
PREFERENCE_BLEND_WEIGHT	0.85	질의 벡터 비율; 사용자 벡터 비율은 0.15
후보 풀 배수	40	PLACE_CANDIDATE_POOL_MULTIPLIER; 최종 limit의 40배를 pgvector 후보로 검색
CRAG_TARGET_CONFIDENCE	0.61	이보다 약하면 외부 폴백 검색 수행
CRAG_MIN_CONFIDENCE	0.52	최종 accept 게이트
앵커 반경 단계	2/5/10 km	후보 구성 하한을 만족할 때까지 확장
식음 후보 상한	0.375	음식점+카페가 후보 풀을 과점하지 않게 제한; 하한이 상한보다 우선
일차별 하한	식당 2D 카페 D	D일 일정의 점심/저녁/휴식 슬롯 확보; 불거리는 최소 2
LLM timeout/temperature	90 s / 0.2	OpenAI-compatible planner 호출
결정적 재생성	최대 3회	근접 후보 시드를 이동하며 제약 통과 안을 탐색

3.5. AI 일정 생성과 제약 검증

3.5.1. 일정 생성 알고리즘

Planner는 여행, 취향, 기존 일정, 일차별 지역을 조회한 뒤 후보와 예보를 수집한다. 단일 지역은 LLM이 후보를 배치하고, 일차별 지역이 다르면 각 지역의 경계를 보장하기 위해 결정적 배치를 사용한다. LLM 프롬프트에는 후보 ID, category, address, opening hours, CRAG confidence, 여행 날짜, 일차별 시작 시각, 목표 항목 수, 취향과 날씨 힌트가 들어간다. 모델 출력에서 후보 ID가 없거나 day/order 범위를 벗어나거나 같은 후보/slot이 중복되면 해당 행을 버린다.

LLM이 일부만 유효하게 반환하면 전체 결과를 폐기하지 않고 유효한 선택을 유지한 채 빈 slot만 보완한다. 점심·저녁은 restaurant, 오후 휴식은 cafe, 나머지는 attraction을 우선하는 fillDaySlots 규칙을 적용한다. 우선 일차는 실내 태그 후보를 먼저 선택한다.

Listing 3.1. PlannerService 핵심 흐름의 의사코드

```

trip, preference, existing = load(tripId)
planDays = resolveTargetDays(request.targetDays)
anchors = preserveFinishedItemsAndStartFromNow(planDays)
candidates = retrieveByRegionOrPerDay(CRAG, weather, preference)
draft = LLM.plan(candidateIdsOnly) or deterministicSlotFallback()
draft = enforceMustIncludePlaces(draft)

if validateHardConstraints(draft) and noScheduleShortfall(draft):
    saveOnlyTargetDays(draft, preserveUserMemos=True)
else:
    for attempt in 1..3:
        rebuilt = proximityOrderedDeterministicPlan(attempt)
        if validateHardConstraints(rebuilt) and noScheduleShortfall(rebuilt):
            saveOnlyTargetDays(rebuilt, preserveUserMemos=True)
            return
    saveBestConstraintValidShortPlanOrRejectWithoutOverwriting()

```

오늘 일정의 부분 재계획은 기상 시각부터 다시 짜지 않는다. KST 현재 시각에 10분 준비 여유를 더하고, 현재 방문 중인 항목이 있으면 종료 시각 이후를 앵커로 삼는다. 남은 활동 시간에 45분짜리 장소 하나도 들어가지 않으면 그 일차를 수정 대상에서 제외하여 빈 일정으로 덮어쓰는 일을 막는다.

3.5.2. 하드 제약 엔진

Constraint Engine은 다음 조건을 순서대로 검증한다.

1. **활동 시간 창**: 각 일차의 기상 시각부터 취침 시각까지를 절대 시간 구간으로 계산한다. 취침이 01:00처럼 자정을 넘는 경우도 정상 구간으로 처리한다.
2. **영업시간**: 후보에 영업시간이 있을 때 예정 방문 시각이 폐점 구간인지 검사한다.
3. **이동 가능성**: 같은 일차의 인접 항목마다 경로 ETA를 계산하고, 앞 장소의 종료시각과 다음 장소 시작시각 사이에 ETA가 들어가는지 검사한다.
4. **일차 경계**: 잘못된 날짜로 밀린 항목이 다음 일차와 겹치지 않는지 확인한다.

도보는 하버사인 거리 기반 로컬 추정, 자동차는 Kakao Mobility, 대중교통은 ODsay를 사용한다. 키 누락, 네트워크 오류, quota 초과, 경로 없음은 원인별 metric을 남기고 로컬 ETA로 대체한다. 검증 실패 시 LLM을 다시 호출하지 않고 후보를 하버사인 근접 순으로 재정렬하여 최대 3회 재생성한다. 제약을 통과하지만 하루 끝에 일부가 잘린 안만 남으면 가장 적게 잘린 안을 저장하고 경고를 남긴다. 제약 자체를 통과하는 안이 없으면 예외를 반환하며 기존 일정은 교체하지 않는다.

3.5.3. 결정적 폴백이 실패를 가리는 문제

결정적 폴백은 LLM이 불안정해도 서비스가 멈추지 않게 하지만, 그 자체가 관측을 어렵게 만드는 부작용이 있다. 운영 중 실제로 관측된 사례는 두 가지다.

첫째, **reasoning 모델의 빈 content**이다. gemma-4-26B-A4B-it처럼 thinking을 하는 모델은 OpenAI-compatible 응답에서 결과를 message.reasoning_content에 싣고 message.content를 비운다. Planner는 content만 읽고 비어 있으면 '{}'로 폴백하므로, **LLM이 멀쩡히 응답하고 있는데도 매 요청이 결정적 폴백으로 빠지고 HTTP 오류도 예외도 남지 않는다**. 서빙 스크립트에서 --reasoning-budget 0으로 thinking을 즉시 종료시켜 막았으며, 모델을 교체할 때는 응답의 content가 비어 있지 않은지 확인하는 것이 검증 절차가 된다.

둘째, **타임아웃이 너무 짧을 때도** 같은 모양이 된다. 초기 기본값 12초는 후보 16개 일정 JSON 생성에 필요한 15~40초에 미치지 못해, 추론 서버가 정상인데도 모든 요청이 폴백으로 떨어졌다(§4.7 참조). 기본값을 90초로 올려 해소했다.

두 사례의 공통점은 실패가 에러가 아니라 품질 저하로만 나타난다는 것이다. 폴백을 갖춘 시스템에서는 폴백률 자체를 지표로 남겨야 하며, 이는 §4.8에서 모델 교체 시 재검증 항목으로 다시 다룬다.

3.6. 알림과 재계획

3.6.1. 알림 우선 정책

미도착, 날씨, 혼잡은 자동 재계획 trigger가 아니라 **추천 알림 trigger**이다. 서버 스케줄러가 조건을 감지하면 inbox record와 FCM push를 생성하고, 앱이 열려 있으면 WebSocket으로 인박스 무효화/토스트 이벤트도 보낸다. 사용자가 알림을 열어 Planner 화면에서 범위와 조건을 확인한 뒤 재계획을 요청한다. 이 정책은 AI 결과가 기존 일정을 사용자 동의 없이 덮어쓰지 않게 한다. 그림 3.6은 감지에서 저장까지의 전체 흐름이다.

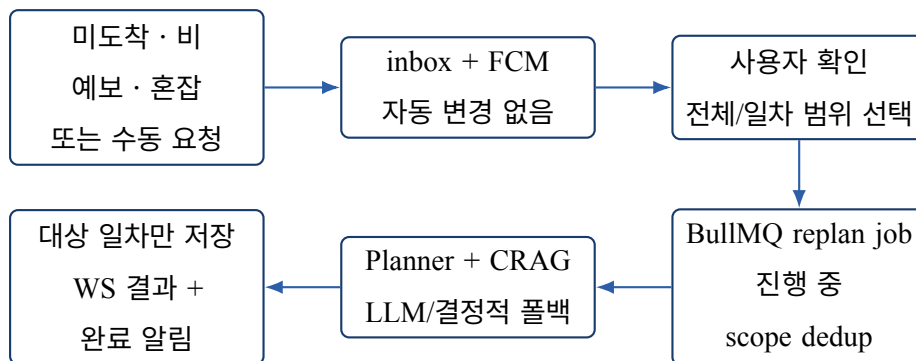


그림 3.6. 알림 확인 후 재계획 흐름

그림 3.7는 이 흐름의 앞 두 단계가 화면에서 어떻게 보이는지를 나타낸다. 왼쪽 인박스는 종류별 필터와 함께 미도착 · 날씨 · 혼잡 알림을 쌓아 두고, 각 알림은 일정을 바꾸는 대신 “일정 변경” 버튼만 제공한다. 이 버튼이 오른쪽 재계획 모달을 연다 — 자연어 요청, 꼭 포함할 장소, 일정 강도, 예산, 피하고 싶은 것을

사용자가 지정한 뒤에야 BullMQ 잡이 등록된다.

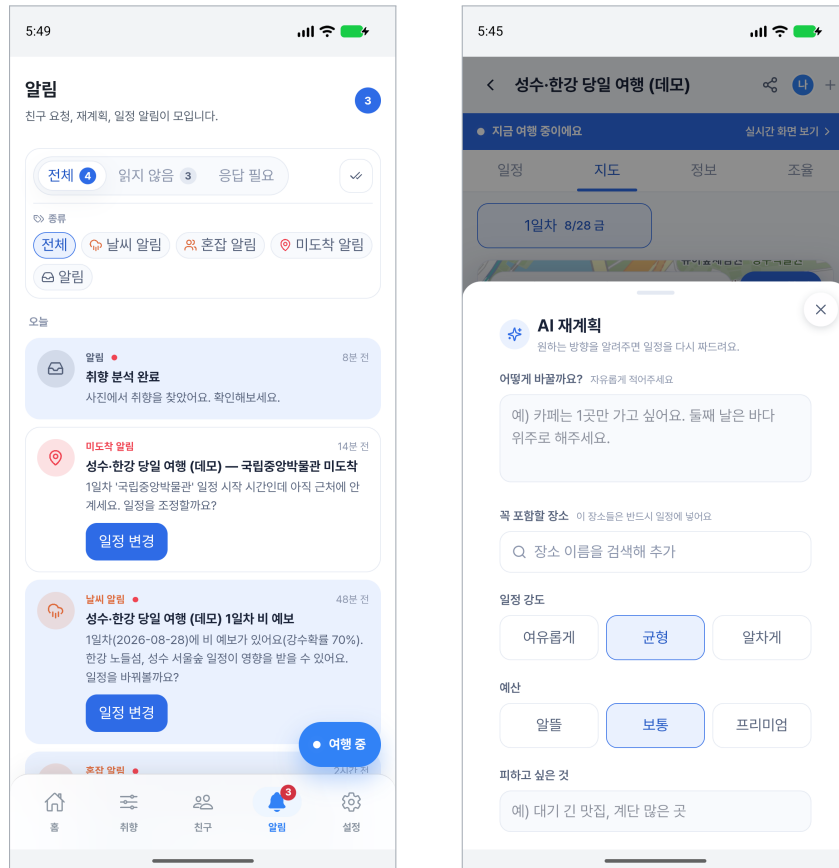


그림 3.7. (좌) 미도착 · 날씨 · 혼잡 알림 인박스, (우) 사용자 확인형 재계획 요청 modal

세 알림은 각각 다른 스케줄과 판정 조건을 가진다. 표 3.4는 코드 상수를 정리한 것으로, 값은 모두 캘리브레이션 대상이다. 미도착 판정은 서버가 하며 클라이언트는 진행 중 위치를 주기 보고할 뿐이다. 마지막 위치 보고가 10분보다 오래되면 “위치를 모르는 상태에서 미도착 처리”를 피하기 위해 판정 자체를 건너뛰고, 시작 +유예를 60분 넘긴 항목은 사용자가 이미 그 일정을 접었다고 보아 알리지 않는다.

표 3.4. 알림 트리거별 스캔 주기와 판정 임계값

알림	스캔 주기	판정 조건과 임계값
미도착	5분마다	항목 시작 시각 + 유예 15분이 지나도 현재 위치가 항목 좌표 반경 500 m 밖. 위치 신선도 10분 이내일 때만 판정(캐시 TTL 15분), 시작 후 60분까지만 대상, (여행 · 사용자 · 일차)당 1회
날씨	예보 발표 10분 후 (02 · 05 · 08 · 11 · 14 · 17 · 20 · 23시)	강수확률 60 % 이상인 시간 슬롯이 2개 이상인 일차. 예보 지평 10일, 야외 영향이 큰 attraction 항목만 대상
혼잡	매일 03:30	예측 집중률이 그 장소 평균의 1.2배 이상이고 절대 집중률 10 이상인 날. 예측 지평 27일, KTO 스캔 호출 예산 300회/일, attraction 항목만 대상

3.6.2. 비동기 잡과 실시간 결과

재계획 요청은 접근 가능한 owner 또는 accepted member만 등록할 수 있다. BullMQ 등록 전 같은 여행에서 대상 일차가 겹치는 active/waiting/delayed 잡을 찾아 중복 요청을 합치며, 10초 bucket job ID로 동시 제출 race를 한 번 더 막는다. Redis가 응답하지 않으면 HTTP 요청이 무한 대기하지 않도록 timeout 후 503을 반환한다. 기본 잡 옵션은 최대 3회, 2초 고정 backoff이다.

Worker는 Planner 결과를 저장한 뒤 trip-session:{tripId} room에 replan_result를 전송하고 inbox/FCM 완료 알림을 만든다. WebSocket 연결은 handshake JWT를 검증하고, join-trip 시 멤버십을 다시 확인한다. 멤버가 제거되면 이미 room에 들어간 소켓도 즉시 퇴장시킨다.

3.7. 데이터 모델과 외부 도구

3.7.1. 핵심 데이터 모델

TypeORM entity 13개와 raw SQL 벡터 테이블 3개가 최종 데이터 모델을 이룬다. 그림 3.8은 핵심 관계만 단순화한 것이다. 사용자 삭제는 관련 여행 · 취향 · 친구 · 알림에 명시적 cascade 또는 set-null 정책을 적용한다.

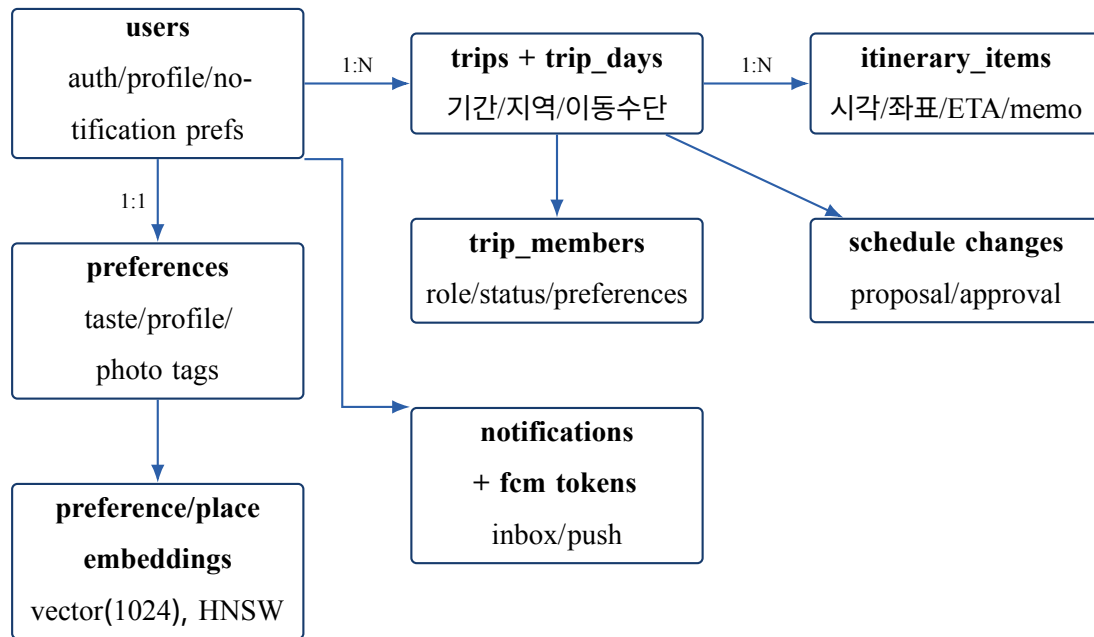


그림 3.8. TriPick 핵심 데이터 관계

표 3.5는 연동한 외부 서비스와 각각의 실패 · 품질 대응을 정리한 것이다.

표 3.5. 주요 외부 API와 코드 사용 위치

서비스	용도	실패/품질 대응
Kakao Local/Map	장소 검색, 목적지 앵커, 지도/주소	지역 hard gate, 좌표 검증, pgvector 우선
Kakao Mobility	자동차 ETA	오류 코드별 metric 후 로컬 거리 추정
ODsay	대중교통 ETA	key/domain/quota/경로 없음 분리 후 폴백
KTO Tour API	관광지/축제/영업시간/지역 코드	일 호출 예산, append cursor, source 품질 gate
KTO 집중률	혼잡 예상 알림	이름/지역 일치 시에만 알림; 랭킹에는 미반영
KMA 단 · 중기예보	일정 날씨와 weather alert	nx/ny 변환, Redis cache, 중기예보 보강[14]
Naver 검색	추천 글의 장소 언급 인지도	커버리지 gate, 소프트 재랭킹, 코퍼스 drift 기록
Firebase FCM	달한 앱 push	토큰 정리, 사용자 카테고리별 수신 설정[15]

3.8. 보안, 운영 안정성, 관측성

3.8.1. 계정 탈취 경로 차단

인증에서 가장 크게 바뀐 부분은 기능이 아니라 이메일 인증 링크가 무엇을 활성화하는가이다. 코드 리뷰에서 확인한 공격 시나리오 세 가지와 대응은 다음과 같다.

1. **기존 계정에 비밀번호 심기.** 가입 시 비밀번호를 대기 상태로 저장하고 인증 링크로 승격하는 설계는 신규 가입에는 맞지만 이미 있는 계정에 적용하면 뒤집힌다. 그 인증 메일은 계정 주인에게 가고 문구는 “가입해주셔서 감사합니다”이므로, 주인이 자기 계정 메일로 알고 누르면 공격자가 넣은 비밀번호가 켜진다. 따라서 이미 있는 이메일로 가입 요청이 오면 **계정을 전혀 건드리지 않고** 주인에게 상황만 알리는 메일을 보내며, CTA는 “비밀번호 재설정”이다. 기존 계정의 비밀번호 설정 · 변경은 재설정 플로우만 통한다.
2. **이메일 선점.** 위 규칙은 “먼저 신청한 쪽이 이긴다”는 뜻이라 반대 방향이 열려 있었다 — 공격자가 피해자 이메일로 먼저 가입해 대기 비밀번호를 심어 두면, 피해자가 나중에 진짜로 가입하고 자기 링크를 누르는 순간 공격자의 비밀번호가 활성화된다. 원인은 대기 비밀번호를 계정에 한 칸으로 두어 여러 신청이 그 칸을 두고 다툰 것이다. 대기 비밀번호와 닉네임을 계정이 아니라 **인증 토큰**(`email_tokens.pendingPasswordHash / pendingNickname`)에 실어, 누른 링크가 담고 있던 값만 켜지도록 바꿨다. 승격은 계정에 비밀번호가 없을 때만 일어나고, 재설정 · 카카오 로그인처럼 이메일 소유가 증명되는 시점에는 남은 대기 토큰을 모두 만료시킨다.
3. **IP를 바꿔 가며 한 주소로 메일 몰기.** 라우트의 `@Throttle`은 IP 기준이라 IP를 갈아 가며 특정 주소로 메일을 반복 발송시킬 수 있다. 이를 막는 수신 주소 기준 한도 (`EmailSendLimiterService`, 5회/시간)가 재발송에는 있고 가입에는 없어 `/auth/signup` 으로 그대로 우회되었다. 가입에도 같은 한도를 붙이되 **버킷을 verify로 공유**하는 것이 핵심이다 — 가입용 버킷을 따로 두면 합계 한도만 두 배가 되고 우회는 그대로다.

이 셋의 공통 원칙은 응답과 부작용이 계정 존재 여부에 따라 갈리지 않게 하는 것이다. 가입은 계정 유무와 무관하게 항상 bcrypt 해싱을 수행하고(분기에 따라 응답이 빨라지면 그 시간차가 곧 가입 여부 조회 API가 된다), 한도 소모도 실제 발송 여부와 무관하게 일어난다. refresh 토큰은 조건부 UPDATE로 회전 승자를 원자적으로 가리고, 30초 유예 안의 재사용은 탈취가 아니라 클라이언트 재시도로 보아 해당 요청만 거절한다. 프로덕션에서 기본 JWT secret이나 발송 불가능한 이메일 설정을 쓰면 부팅을 거부한다 — 후자는 응답 200에 메일 0통, 에러 로그도 없는 조용한 사고이기 때문이다. 상세한 근거와 잔여 위험은 저장소의 인증 하드닝 문서에 기록하였다.

3.8.2. 운영 안정성과 관측성

최종 구현은 기능 외에 운영 실패 경로를 코드로 다룬다.

- HTTP/WS CORS는 같은 allow-list를 사용하고, Express 식별 헤더 제거와 공통 보안 헤더를 적용한다.
- JWT algorithm을 고정하고 WebSocket도 access token과 여행 접근 권한을 검증한다.
- BullMQ worker는 SIGTERM 시 진행 중 잡을 마친 뒤 종료해 재배포 중 중복 실행을 줄인다.
- Redis는 REDIS_URL의 password/TLS를 지원하고, cache · queue · throttling 연결 설정을 공통 함수로 관리한다.
- 개발에서는 TypeORM synchronize, 그 외 환경에서는 migration만 실행해 스키마 경로를 배타적으로 유지한다.
- Sentry는 API 예외, BullMQ worker 오류, Next.js 소스맵을 분리 관측하며, health endpoint는 외부 의존성 일시 장애로 재시작이 반복되지 않도록 liveness만 확인한다.

- 이미지 저장소는 private bucket과 presigned URL을 사용하며, R2 전환은 객체 이전 후 DB key migration 순서를 따른다.

4. 연구 결과 분석 및 평가

4.1. 개발 수행 체계와 역할 분담

본 과제는 2인 팀이 2026년 5월 4일 첫 커밋부터 8월 31일 최종 코드까지 약 17주 동안 수행하였다. 저장소는 develop을 기준 브랜치로 두고 기능 단위 feat/* 브랜치를 파서 Pull Request로 병합하는 방식을 유지했으며, 이 기간 동안 140개의 Pull Request가 병합되고 694개의 커밋이 기록되었다.

역할은 화면과 서버 도메인을 축으로 나누었다. 표 4.1은 저장소의 커밋 이력에서 각 구성원이 실제로 작성 · 수정한 경로를 집계해 정리한 것이다.

표 4.1. 구성원별 담당 영역

구성원	주 담당	주요 산출물
고태영	AI 파이프라인, 서버 도메인, 데이터 · 인프라	Planner 오케스트레이션과 제약 엔진, pgvector 적재 · CRAG 검색과 평가 하네스, 인증 하드닝, 알림 3종 (미도착 · 낱씨 · 혼잡) 스케줄러, BullMQ · WebSocket, RunPod 추론 서빙, Railway/Vercel/R2 배포와 운영 문서
박준이	웹 프론트엔드와 디자인 시스템, 협업 도메인	랜딩 · 로그인 · 여행 생성 · 플래너 화면 UI, 공용 UI 컴포넌트와 비주얼 리디자인, 취향 입력 · 취향 조율 화면, 친구 · 여행 멤버 · 초대 기능, 모바일 앱 아이콘과 스토어 리소스
공통	설계 합의와 검증	요구사항 조정, packages/types 공유 DTO 규약, 화면 · API 계약, PR 리뷰, 보고 단계별 문서화

착수보고서의 WBS는 취향 분석, 일정 생성, 실시간 대응, 추론 인프라, 통합의 다섯 묶음이었다. 표 4.2는 각 묶음이 실제로 코드에 처음 들어온 시점과 계획 대비 차이를 정리한 것이다. 시점은 해당 디렉터리의 최초 커밋 날짜이다.

표 4.2. WBS 대비 실제 수행 일정

작업 묶음	실제 착수	계획 대비 차이
인증 · 기본 도메인 · 모바일 셀	2026.05.04	계획대로 1주차 착수. 인증은 6~8월에 하드닝으로 두 차례 재작업
취향 분석 파이프라인	2026.05.04	골격은 계획대로였으나, 수집 경로가 Instagram Graph API에서 갤러리 직접 업로드로 바뀌며 중간 단계에서 재설계
일정 생성 · 제약 엔진	2026.05.04	Planner와 Constraint Engine은 초기부터 존재했고, 후보 품질 문제가 드러난 뒤 7~8월에 결정적 재생성과 부분 재계획을 추가
협업(친구 · 멤버 · 조율)	2026.05.17	착수 WBS에 없던 항목. 동행자 취향 조율 요구가 나오면서 추가된 범위
pgvector 적재 · CRAG 검색	2026.06.30	계획보다 늦게 착수한 최대 난관. 중간보고 이후 8월까지 카탈로그 재적재와 랭킹 튜닝이 반복되어 전체 일정의 병목이 되었다
실시간 알림 3종	2026.07.18~07.20	착수 계획의 “자동 재계획”이 “알림 후 사용자 승인”으로 바뀌면서 날씨 · 미도착 · 혼잡 세 스캐너로 재구성
검색 평가 하네스	2026.07.27	착수 계획의 LLM 시나리오 harness 대신 검색 골든셋 평가로 대체 (§4.5 참조)
배포 · 통합	2026.08	계획대로 최종 단계. Railway/Vercel/RunPod/R2 구성과 운영 문서화

일정 운영에서 얻은 교훈은 두 가지다. 첫째, **검색 품질 작업의 소요를 과소평가했다.** 착수 WBS는 pgvector 적재를 단일 작업으로 잡았으나, 실제로는 적재 · 평가 · 재튜닝이 맞물린 반복 루프였고 8월 말까지 이어졌다. 둘째, **요구사항 변경이 일정보다 설계에 더 큰 영향을 주었다.** 자동 재계획을 알림 우선으로 바꾼 결정은 구현량 자체는 비슷했지만 알림 · 승인 · 부분 재계획이라는 새 흐름을 요구했고, 이것이 7월 이후 작업의 골격이 되었다.

4.2. 착수 · 중간 · 최종 요구사항 추적

표 4.3는 착수보고서의 핵심 요구를 기준으로 중간보고서의 상태와 최종 결과를 연결한 추적표이다. “완료”는 최종 코드와 테스트 · 운영 기록이 모두 있는 경우, “부분 완료”는 핵심 코드는 있으나 실기기 · 외부 서비스 · 장시간 부하와 같은 운영 검증이 남은 경우, “변경”은 동일한 사용자 문제를 다른 흐름으로 해결한 경우로 구분하였다.

표 4.3. 착수-중간-최종 요구사항 추적표

요구 영역	착수 목표	중간보고 상태	최종 판정	최종 근거
취향 프로필	사진/텍스트에서 취향 tag와 embedding 생성	수동 입력 · 직접 업로드 · 재임베딩 구조 구현	완료	구조화 취향, private 사진, BullMQ 분석, tag on/off, 1,024차원 preference embedding
장소 카탈로그	관광/지도 API 결과를 검색 근거로 사용	적재 CLI와 pgvector schema는 구현, 실환경 대량 적재 필요	완료	라이브 71,684행, KTO 33,927 · Kakao 37,757, HNSW, 축제 기간 915건
CRAG 검색 보정	confidence 평가 · 재검색 · reranking	evaluator, fallback, 취향 blending 구현	완료	6항 점수, 지역/앵커 검색, quota, 식음 상한, 16개 검색 평가 케이스
일정 생성/제약	JSON 일정과 수면 · 영업 · 이동 hard constraint	Planner/Constraint Engine 구현, fixture 보강 필요	완료	후보 ID 제한, 절대 시간창, opening hours, 인접 ETA, 최대 3회 결정적 복구, 유효안 없으면 저장 방지
실시간 재계획	웨이팅 · 이탈 · 우천 시 대안 push	Live/WebSocket은 부분 구현, E2E · 실기기 검증 필요	변경 · 완료	날씨 · 혼잡 · 미도착 알림과 사용자 승인형 전체/일차 재계획, BullMQ · WS · FCM 결과 전달
LLM harness	prompt/tool/schema/fixture 회귀 관리	OpenAI-compatible adapter와 hardening test 구현	부분 완료	unit 997개와 검색 골든셋은 확보했으나 최초 20개 scenario pass-rate 정의와 LLM nightly 환경은 미구축

다음 쪽에 계속

표 4.3. 착수-중간-최종 요구사항 추적표(계속)

요구 영역	착수 목표	중간보고 상태	최종 판정	최종 근거
모바일 셀	React Native WebView, 위치, push	브리지 · foreground service 후보 구현, 실제 기기 검증 필요	부분 완료	arm64 debug/iOS simulator 기록, 위치 · FCM · Keychain bridge 구현; 장시간 실기기 · 배터리 · APNs 전달률 미검증
추론 인프라	llama.cpp에서 vLLM/Triton으로 단계 확장	adapter로 endpoint 교체 가능하도록 추상화	변경 · 운영	RunPod llama.cpp 2-process와 결정적 fallback 운영; vLLM 동시성 · Triton batch 목표는 미평가
통합/문서화	재현 가능한 demo와 보고서	typecheck/test 기준선과 기능 문서 다수 작성	완료	77개 기능 · 운영 문서, 15 개 migration, 배포 가이드, 6 suites/81 DB e2e 기록

중간보고서에서 명시한 잔여 작업 중 장소 적재 · 취향 재임베딩, CRAG threshold 조정, 제약 실패 복구, WebSocket 재계획 결과, FCM production 배선, 최종 문서화는 최종 단계에서 코드와 기록으로 종결되었다. 반면 Android/iOS 실제 기기의 장시간 위치 추적과 전체 외부 의존성을 포함한 상시 통합 테스트는 완료 근거가 부족하므로 후속 검증으로 남겼다.

4.3. 최종 구현 결과 요약

표 4.4는 최초 목표와 최종 코드 결과를 비교한다. “구현”은 단순 UI 자리표시자가 아니라 service/controller/entity 또는 네이티브 브리지와 테스트가 존재하는 경우로 판정하였다.

표 4.4. 핵심 목표별 최종 구현 결과

영역	상태	최종 구현 근거
이메일/Kakao 인증	구현	이메일 인증 · 재설정, Kakao state/exchange, refresh rotation, 모바일 Secure Storage
취향 수동 입력	구현	음식 · 무드 · 환경 · 속도 · 활동 · 혼잡 프로필과 공유 DTO
목적지 추천	구현	취향 벡터 기반 지역 랭킹 + 계절 코퍼스 결합(0.7:0.3), 신호 결손 시 단계적 중립 폴백
사진 취향 분석	구현	private upload, BullMQ worker, Vision tags, 사진별 tag on/off, 재임베딩
장소 RAG/CRAG	구현	4종 적재 source, pgvector/HNSW, 지역/앵커 검색, 6항 평가, 외부 폴백
일정 생성/검증	구현	후보 ID 제한 LLM, slot 보완, 활동/영업/ETA 제약, 최대 3회 결정적 재생성
재계획	구현	전체/일차 범위, 오늘 앵커, must-include, pace/budget/avoid, in-flight dedup
실시간 알림	구현	미도착 · 날씨 · 혼잡 inbox/FCM, WebSocket invalidate/result
협업	구현	친구, 여행 멤버, 초대, 취향 조율, 일정 변경 승인, 공유 링크
웹/모바일	부분 완료	Next.js 웹 19개 page route와 React Native 위치/FCM/Keychain bridge 구현; 장시간 실기기 위치 · 푸시 검증은 잔여
프로덕션 배포	구현	Vercel/Railway/RunPod/R2 구성, migration, Docker, Sentry, health check
Instagram 자동 수집	변경	플랫폼 심사 의존으로 직접 업로드+수동 입력을 정본 플로우로 채택
vLLM/Triton 서빙	유보	RunPod llama.cpp chat/vision · embedding 2-process로 운영하고 adapter 호환성 유지
다중 API replica	제한	Socket.IO Redis adapter 미도입; 현재 replica 1개 운영

4.4. 코드 품질과 빌드 검증

최종보고서 작성 시점에 lockfile을 유지한 채 pnpm 의존성을 동기화하고 테스트를 재실행하였다. Watchman이 없는 환경이므로 Jest에 `--no-watchman --runInBand`를 지정하였다. 표 4.5의 현재 검증과 저장소 문서의 DB e2e 기록을 구분한다.

표 4.5. 최종 코드 검증 결과

검증	결과	범위/비고
API Jest unit	73 suites, 928 tests 통과	Planner, CRAG, 적재, auth, storage, alerts, main-planner 등
Utils Jest unit	7 suites, 69 tests 통과	날짜, 활동 시간 창, 지리, 격자 변환, 단 · 중기예보 parser
Workspace typecheck	5/5 통과	API, Web, Mobile, types, utils의 tsc --noEmit
Production build	4 tasks 통과	API, Web, types, utils; Next.js optimized build 포함
DB e2e 기록	6 suites, 81 tests 통과	2026-08-27 문서 기준; Postgres가 필요한 경량 e2e 3회 재현
AI integration	수동 분리	PostgreSQL, Redis, chat(:8080), embedding(:8081)이 모두 필요해 CI 제외

API unit test는 총 928개의 assertion 경로를 통과했다. 이 중 Planner 테스트는 AI 초안 제약 위반, 결정적 재생성 복구, 일자별 지역, 부분 재계획, 오늘 시각 앵커, 후보 부족 경고, 우천 실내 우선, 경로 API 폴백을 다룬다. CRAG 테스트는 지역 정합, 인지도 coverage gate, 취향 종립 처리, category 하한/상한, 근접 중복 제거를 검증한다. 빌드는 성공했으며 Next.js의 metadataBase 미설정 경고가 남아 있으나 생성물 실패는 아니다.

4.5. 착수 단계 평가 목표 대비 달성도

착수보고서는 구현 전 단계에서 정량 목표를 넓게 설정하였다. 최종 단계에서는 데이터와 코드가 축적되면서 일부 지표의 정의가 바뀌었고, 사용자 연구나 동시 부하 환경이 필요한 항목은 측정하지 못했다. 서로 다른 지표를 억지로 같은 것으로 보지 않기 위해 표 4.6에 최초 목표, 최종 관측, 판정을 분리하였다.

표 4.6. 착수보고서 평가 목표와 최종 관측 비교

평가 항목	착수 목표	최종 관측	판정
일정 실행 가능성	30개 scenario에서 hard constraint 위반 0건	Constraint/Planner 단위 테스트와 DB e2e, 검색 하네스의 영업시간 밖 후보 0건을 확인했으나 동일한 고정 30개 집합 결과는 없음	부분 달성

다음 쪽에 계속

표 4.6. 착수보고서 평가 목표와 최종 관측 비교(계속)

평가 항목	착수 목표	최종 관측	판정
취향 반영도	5점 척도 평균 4.0 이상	preference blending과 profile별 후보 차이는 코드로 검증했으나 실제 사용자 5점 설문은 미수행	미평가
검색 근거성	낮은 confidence 후보의 90% 이상 보정 또는 제거	정의를 Recall@5/10, R cat, MRR, 지역정합, 금지어로 변경; 최종 Recall@10 0.407, MRR 0.734, 지역정합 100%	지표 변경
재계획 응답 속도	이벤트 입력부터 대안 push까지 15초 이내	LLM 단계만으로 후보 16개 JSON 생성에 15~40초(warm)로 목표를 이미 초과. 검색 · 경로 · queue · WS를 포함한 end-to-end p95는 미측정	미달 · 부분측정
LLM 출력 안정성	1회 재시도 내 schema 성공률 95% 이상	JSON/schema 오류, timeout, 제약 실패와 결정적 fallback을 테스트했으나 운영 성공률 집계는 없음	부분 달성
Harness 회귀 안정성	주요 scenario 20개, pass rate 90% 이상	16개 검색 케이스와 997개 unit test를 운용하지만 최초의 통합 pass-rate 정의와 동일하지 않음	지표 변경
vLLM 추론 성능	동시 요청 5개 기준 p95 10초 이내	OpenAI-compatible adapter는 유지했으나 최종 운영 서버는 llama.cpp이며 동시성 실험은 미수행	유보
Triton 보조 모델	후보 20개 embedding/reranking 3초 이내	embedding은 llama.cpp 별도 포트로 운영; Triton endpoint와 batch 20 실험은 미구현	유보

이 결과는 최초 목표를 모두 달성했다고 포장하기보다, 최종 시스템에서 실제로 검증한 범위를 명확히 한다. 특히 검색 품질은 단일 “보정률” 대신 카탈로그 결손과 랭킹 실패를 분리하는 R|cat, MRR, 적재 커버리지로 발전했다. 반면 사용자 만족도, 전체 재계획 지연, 동시 추론 성능은 별도 실험 설계가 필요한 미평가 항목이다.

4.6. 검색 품질 평가

프로덕션 카탈로그는 2026년 8월 재적재에서 6,473행에서 71,684행으로 확대되었다. KTO 33,927행, Kakao 37,757행이며, 축제 기간 915건과 category detail이 보강되었다. 평가 하네스는 16개 목적지 · 취향 · trigger 케이스를 사용하며, 케이스당 검색 결과 수는 16개이다. 모든 지표는 케이스마다 계산한 뒤 16개 케이스에 대해 산술평균한 macro 평균이다. 지표 정의는 다음과 같다.

- **Recall@k**: 골든 정답 중 상위 k 후보에 포함된 비율
- **R|cat@16**: 카탈로그에 실제 존재하는 정답만 분모로 삼고, 반환된 후보 풀 전체(16개)에서 계산한 recall. 카탈로그 결손을 제외한 랭킹 자체의 성적이다
- **적재 커버리지**: 골든 정답 중 카탈로그에 적재되어 있는 비율
- **MRR**: 첫 정답의 역순위 평균
- **지역정합**: 목적지 지역과 최종 후보 지역이 일치하는 비율

R|cat@16은 분모(카탈로그 적재분)와 절단 지점($k = 16$)이 모두 Recall@10과 다르므로, 표 4.7의 0.486을 Recall@10을 커버리지로 나눈 값과 직접 비교할 수 없다.

표 4.7. 프로덕션 검색 최종 기준선

지표	결과	지표	결과
Recall@5	0.248	Recall@10	0.407
R cat@16	0.486	MRR	0.734
적재 커버리지	98%	지역정합	100%
금지어 후보	0	영업시간 밖 후보	0

초기 10,721행 카탈로그를 전국 40,304행으로 늘렸을 때 커버리지는 74%에서 97%로 상승했지만, 같은 후보 풀 배수에서는 경쟁이 증가해 R|cat과 MRR이 낮아졌다. 이를 통해 카탈로그 확대가 곧바로 랭킹 향상을 의미하지 않음을 확인하였다. 후보 풀 배수, 인지도 coverage gate, 취향 풀백 중립, 식음 상한, 근접 중복 규칙을 다시 조정해 현재 기준선을 확정했다.

프로덕션과 로컬의 카탈로그 구성은 동일하지 않다. 프로덕션은 Kakao 행이 로컬보다 약 3배 많아 식음 후보 경쟁이 더 강하며, 서로 다른 시점의 Naver 코퍼스가 섞여 직접 A/B로 단정할 수 없다. 문서화된 변동 폭은 Recall@10 약 ± 0.013 이다. 따라서 표 4.7은 현재 프로덕션의 재현 가능한 기준선으로 사용하되, 모델 · 카탈로그 · 외부 코퍼스가 바뀌면 다시 측정해야 한다.

4.7. 실행 성능과 시스템 검증

RunPod의 RTX PRO 4000 Blackwell 24GB(secure, EU-RO-1) 환경에서 chat/vision 모델 gemma-4-26B-A4B-it(Q4)와 embedding 모델 BGE-m3-ko(f16)를 한 pod에 실행한 기록은 표 4.8과 같다. 추론 서버가 정상일 때 110 token 생성에 약 2.5초가 걸렸으며, 로컬 LLM 실패 시에는 Planner가 timeout/예외를 감지해 결정적 폴백으로 일정을 반환한다.

다만 이 2.5초는 짧은 생성의 예시일 뿐 재계획 1회의 비용이 아니다. 실제 플래너 호출은 후보 16개를 받아 일정 JSON 전체를 만들어야 하므로 warm 상태에서도 15~40초가 걸린다. 기본 타임아웃이 12초에서 90초로 올라간 것이 이 실측 때문이다 — 12초에서는 LLM이 살아 있어도 매번 결정적 폴백으로 떨어졌다. 따라서 착수보고서의 “이벤트 입력부터 대안 push까지 15초” 목표는 LLM 단계 하나만으로도 이미 초과하며, 검색 · 경로 조회 · 큐 대기 · WebSocket 전달을 포함한 end-to-end p95는 별도 측정이 필요하다. 사용자 승인형 정책에서는 재계획이 화면을 막지 않고 백그라운드 잡으로 돌아 결과를 push하므로 이 지연이 곧바로 사용성 문제로 이어지지는 않지만, 목표 수치 자체는 재정의되어야 한다.

표 4.8. 추론 및 배포 실측

항목	실측	해석
첫 기동(17GB 모델 다운로드)	약 70분	네트워크 volume 미사용 시 재시작 비용이 큼
volume cache 재기동	약 40초	상시 pod에서 운영 가능한 수준
chat 생성	110 token / 2.5초	정상 warm 상태의 예시; cold start는 별도
embedding	1,024차원, L2 norm 1.0	DB vector 차원과 일치
의미 유사도 예시	카페-커피숍 0.68	카페-등산로 0.29보다 높아 의미 방향 확인
Docker API 검증	health/write 200	빈 DB migration, 17개 table, HNSW index 생성 확인

2026년 6월의 브라우저 E2E 기록에서는 시작 화면에서 여행 생성, Planner, 당시의 waiting 재계획까지 이어졌고, 부산 일정 4개가 CRAG/LLM으로 생성된 뒤 WebSocket cache refresh로 결과가 반영되었다. 이후 신뢰할 수 있는 waiting 데이터원이 없다는 판단으로 해당 trigger는 제거하고 관광공사 혼잡, 날씨, 미도착 알림과 사용자 승인형 재계획으로 변경했다. 따라서 과거 E2E는 계층 간 연결 근거로는 사용하되, 최종 이벤트 정책 전체의 재검증으로 간주하지 않는다.

모바일 QA에서는 Android arm64 debug build, iOS pod install/simulator build, 390x844 WebView 화면을 확인하였다. 다만 Android foreground service의 화면 꺼짐 · 앱 종료 이후 장시간 위치 추적, iOS Always 권한, 실제 FCM/APNs 전달률, 갤러리 업로드는 실기기에서 충분히 검증되지 않았다.

4.8. 한계와 타당성 위험

평가 결과를 해석할 때 다음 한계를 함께 고려해야 한다.

1. **골든셋 규모**: 검색 평가는 16개 케이스에 집중되어 전체 국내 목적지를 대표하지 않는다. 특히 개인화 만족도 자체보다 유명 장소 회수에 가중된 정답 집합이다.
2. **외부 코퍼스 drift**: Naver 검색 결과와 KTO/Kakao 데이터가 시간에 따라 바뀌므로 다른 날짜의 절대 지표 비교에는 drift가 포함된다.
3. **영업시간 커버리지**: Kakao Local은 영업시간 필드를 주지 않아 전체 카탈로그의 영업시간 coverage가 낮다. 데이터가 있는 후보에는 hard constraint가 동작하지만, 데이터가 없는 후보의 실제 휴무를 보장할 수 없다.
4. **AI integration 환경**: 현재 unit/typecheck/build는 재실행했지만, 로컬 LLM과 DB가 모두 필요한 통합 테스트는 CI에서 분리되어 있다. 모델 교체 시 JSON 형식과 응답 지연을 재검증해야 한다.
5. **단일 replica**: Socket.IO Redis adapter가 없어 API를 2개 이상으로 늘리면 room broadcast가 인스턴스 간 전파되지 않는다. 현재는 replica 1개라는 운영 제약으로 해결한다.
6. **알림 정확도**: 미도착은 위치 freshness와 반경 threshold, 혼잡은 관광지 이름 매칭에 의존한다. 잘못된 자동 변경을 피하기 위해 알림 후 수동 재계획 정책을 유지한다.
7. **실기기 검증**: 모바일 네이티브 bridge와 build 기록은 있으나 Android/iOS의 장시간 백그라운드 위치, 배터리 소모, 실제 push 전달률, 사진 chooser를 다기종 실기기에서 검증하지 못했다.
8. **최초 평가 목표의 미측정**: 사용자 만족도 4.0/5, vLLM 5개 동시 요청, Triton batch 20 목표는 현재 결과와 직접 비교할 실험 로그가 없다. 재계획 15초 목표는 LLM 단계만으로 15~40초가 관측되어 미달이 확인되었으나, 검색 · 큐 · 전달을 포함한 end-to-end p95는 여전히 측정되지 않았다.

5. 결론 및 향후 연구 방향

5.1. 결론

본 연구는 사용자 취향과 실시간 여행 맥락을 반영하는 AI 여행 플래너 TriPick을 계획 수준이 아닌 통합 소프트웨어로 구현하였다. 최종 시스템은 취향 입력 · 사진 분석, 여행 생성, 전국 장소 카탈로그, pgvector 검색, CRAG 후보 평가, OpenAI-compatible LLM 일정 배치, 하드 제약 검증, 결정적 폴백, 일차별 재계획, 알림, 협업, 웹 클라이언트, 모바일 native bridge와 프로덕션 배포 구성을 포함한다.

착수보고서의 목표와 비교하면 취향-검색-생성-검증-재계획의 핵심 파이프라인은 구현되었고, 중간보고서에서 잔여 작업으로 남긴 프로덕션 장소 적재, CRAG threshold 조정, FCM 결과 배선, DB e2e와 문서화도 완료하였다. 구현 과정에서 Instagram 자동 수집은 직접 업로드로, 자동 waiting 재계획은 신뢰 가능한 혼잡 · 날씨 · 미도착 알림 뒤 사용자 승인형 재계획으로, vLLM/Triton 계획은 RunPod llama.cpp 2-process 운영으로 변경하였다. 이 변경은 계획 누락을 숨긴 것이 아니라 외부 심사 · 데이터 신뢰성 · GPU 자원 제약에 대응한 최종 설계 결정이다.

핵심 성과는 LLM 생성 품질을 신뢰성의 유일한 근거로 두지 않은 점이다. 후보 ID 제한으로 장소 환각을 차단하고, 지역 pre-filter · CRAG 점수 · 카테고리 균형으로 입력 후보를 보정했다. 생성 후에는 활동 시간, 영업시간, 이동 ETA를 검사하고, 실패하면 근접 후보 기반 결정적 재생성을 수행한다. 이벤트가 발생해도 일정을 자동 덮어쓰지 않고 사용자 확인 후 대상 일차만 교체하며, 기존 메모와 완료 항목을 보존한다.

검증 결과 API와 공통 유틸의 unit test 997개가 모두 통과했고, 5개 workspace typecheck와 API/Web production build가 성공했다. 프로덕션 장소 카탈로그는 71,684행, 적재 커버리지 98%, 지역정합 100%, Recall@10 0.407, MRR 0.734의 기준선을 확보하였다. 이 수치는 완전한 추천 품질의 종결점은 아니지만, 데이터 확대와 랭킹 튜닝을 코드 · 로그 · 골든셋으로 반복할 수 있는 기반을 마련했다. 한편 사용자 만족도, 전체 재계획 p95, vLLM/Triton 부하, 모바일 장시간 실기기 동작은 측정하지 못했으므로 최종 달성으로 주장하지 않는다.

5.2. 연구 기여

본 과제의 구현 기여는 다음과 같다.

- 목적지 추천부터 일정 배치까지 같은 취향 임베딩을 쓰되, 신호가 빠지면 단계적으로 중립으로 물러나는 개인화 경로
- 행정구역과 세부 목적지를 함께 다루는 지역/좌표 앵커 기반 적응형 검색
- 벡터 유사도, 취향, 인지도, 지역성, trigger, 가용성을 결합한 서비스형 CRAG 평가기
- 카테고리 하한과 식음 상한을 함께 적용해 여행 일정 구성 가능성을 보장하는 후보 풀

- LLM의 유효 선택은 살리고 빈 slot만 보완하는 부분 수용 방식
- 하드 제약 실패 시 LLM 재호출 없이 근접 후보로 복구하는 결정적 재생성
- 오늘 시각, 완료 항목, 사용자 메모, 비대상 일차를 보존하는 부분 재계획
- 웹 · 모바일 · API · AI · 데이터를 같은 타입과 테스트로 관리하는 모노레포 운영 구조

5.3. 향후 연구 방향

첫째, 검색 평가는 16개 골든 케이스에서 실제 사용자 평가로 확장해야 한다. 유명 장소 회수와 개인 취향 만족도는 다른 목표이므로, 목적지 대표성, 취향 적합성, 이동 효율, 설명 신뢰도를 분리한 online/offline 평가를 설계할 필요가 있다. RAGAS 방식의 context relevance와 faithfulness를 일정 JSON과 설명 문장에 맞게 적용하는 것도 후속 과제이다[5].

둘째, 카탈로그 밀도와 category 편향을 지속적으로 관측해야 한다. 적재량이 커지면 후보 풀 배수와 인지도 coverage gate의 의미가 달라지므로, 배포 후 자동 회귀 평가와 지역별 분포 dashboard를 구축해야 한다. 영업시간은 KTO 행부터 우선 backfill하고, 데이터가 없는 장소는 사용자에게 확인 필요 상태를 명시하는 방식을 검토한다.

셋째, 다중 replica 운영을 위해 Socket.IO Redis adapter를 도입하고, migration을 API 부팅에서 독립 배포 단계로 분리해야 한다. queue, WebSocket, DB를 포함한 경량 e2e는 CI에 유지하고, LLM/embedding이 필요한 integration suite는 고정 모델 서버가 있는 별도 nightly 환경에서 실행하는 것이 적절하다.

넷째, 모바일 실제 기기의 장시간 위치 추적, 백그라운드 정책, 배터리 사용량, FCM/APNs 전달률을 측정해야 한다. 알림 threshold는 false positive/false negative를 기록해 조정하되, 자동 재계획이 아닌 사용자 확인 정책은 유지한다.

마지막으로 모델 교체 가능성을 높이기 위해 chat/vision/embedding 모델 버전, 프롬프트, JSON schema, 벡터 차원, 카탈로그 embedding hash를 하나의 release manifest로 관리해야 한다. 이 과정을 통해 TriPick은 현재의 졸업과제 결과물을 넘어, 검색 품질과 운영 신뢰성을 반복 개선할 수 있는 실제 AI 여행 서비스로 발전할 수 있다.

6. 참고 문헌

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [2] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, *Corrective retrieval augmented generation*, 2024. arXiv: 2401.15884 [cs.CL].
- [3] pgvector Contributors, *pgvector: Open-source vector similarity search for PostgreSQL*, <https://github.com/pgvector/pgvector>, Accessed August 31, 2026.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.
- [6] Vercel, *Next.js documentation*, <https://nextjs.org/docs>, Accessed August 31, 2026.
- [7] Meta Platforms, *React Native documentation*, <https://reactnative.dev/docs/getting-started>, Accessed August 31, 2026.
- [8] NestJS Team, *NestJS documentation*, <https://docs.nestjs.com/>, Accessed August 31, 2026.
- [9] Taskforce.sh, *BullMQ documentation*, <https://docs.bullmq.io/>, Accessed August 31, 2026.
- [10] Socket.IO Team, *Socket.IO documentation*, <https://socket.io/docs/v4/>, Accessed August 31, 2026.
- [11] TypeORM Contributors, *TypeORM documentation*, <https://typeorm.io/>, Accessed August 31, 2026.
- [12] Korea Tourism Organization, *Korean tourism information service*, <https://www.data.go.kr/data/15101578/openapi.do>, Accessed August 31, 2026.
- [13] Kakao Developers, *Kakao Local API documentation*, <https://developers.kakao.com/docs/latest/en/local>, Accessed August 31, 2026.

-
- [14] Korea Meteorological Administration, *Village forecast information service*, <https://www.data.go.kr/data/15084084/openapi.do>, Accessed August 31, 2026.
 - [15] Google, *Firebase Cloud Messaging documentation*, <https://firebase.google.com/docs/cloud-messaging>, Accessed August 31, 2026.
 - [16] PNU KoZune Team, *TriPick: Preference-aware ai travel planner*, <https://github.com/PNUKoZune/tripick>, Source snapshot e07b47f, August 31, 2026.